
Minigotchi-V3 — Complete Hardware Guide

Firmware: Minigotchi-V3 (dj1ch) · **Upstream:** [dj1ch/minigotchi-ESP32](#) (GPL-3.0) — the **v3.x "V3" release line** (latest tag [v3.6.4-beta](#), Dec 2025). The profile points at [dj1ch/minigotchi-v3](#), which currently returns **404** — verify the canonical repo, see §9. **Chips:** ESP32 (**dual-core, required**), ESP32-S3 · **Cyber Controller profile:** [minigotchi-v3](#) (esptool backend, merged single-`.bin` @ 0x0, GitHub-release resolver, no suicide) **This guide:** purchase → build/config → flash → integrate into Cyber Controller → use → troubleshoot.

1. Overview

Minigotchi-V3 is a Pwnagotchi-inspired ESP32 gadget. Unlike a full Raspberry-Pi Pwnagotchi, it runs entirely on a microcontroller and is deliberately **minimal**: it passively rides nearby WiFi, **hops channels**, sends **deauthentication frames** to provoke 4-way handshakes, and broadcasts its own **advertisement beacon frames** so it can "talk to" other minigotchi and detect/greet nearby Pwnagotchi (a parasite/Palntagotchi-style friend behavior). The captured **WPA handshakes / PMKID** material is what a paired Pwnagotchi cracks — the minigotchi itself is the small, always-on collector/companion. The upstream author is explicit that it is "not your ideal deauthing hacking tool" but a more playful companion device. Hardware is intentionally tiny: a dual-core ESP32 board, optionally a small **SSD1306/SH1106 OLED** (or an M5/CYD/T-Display screen) for a face. Cyber Controller flashes it (single merged `.bin` at offset `0x0`) and manages the serial port; the profile is `minigotchi-v3`.

2. Legal & Safety

Authorized testing only. Minigotchi transmits **deauth frames** and beacons on the 2.4 GHz band, and **handshake/PMKID capture is collection of others' traffic** — both are **illegal against networks/people you do not own or have written permission to test** in most jurisdictions (US: CFAA/FCC; EU and others similar). Run it only against your own lab APs or under an explicit, written engagement scope, and **whitelist** any network you must not touch (see §6/§7). Deauth verbs are treated as dangerous by Cyber Controller and (unless you disable warnings) confirmed before sending. Even passive scanning/sniffing may be regulated where you live — check local law before powering on.

3. Hardware & Purchasing

Minigotchi-V3 needs a **dual-core ESP32** (the profile lists `esp32` and `esp32s3`). **Single-core ESP32 and ESP8266 are NOT supported** — that's a hard upstream requirement. A display is **optional**; the cheapest build is a bare DevKit with no screen, driven over serial by Cyber Controller.

Tier	Board	Chip	Why	Where to buy (search terms)
Cheapest / no screen	ESP32-WROOM-32 DevKit ("DevKit V1")	esp32 (dual-core, 4MB)	Profile's "ESP32 Dual-Core Generic"; serial-only via Cyber Controller	AliExpress/Amazon: "ESP32 DevKit V1 WROOM-32"
Add a face (cheap)	DevKit/WROOM + SSD1306 / SH1106 0.96" I²C OLED	esp32	Tiny pet face; wire 4 pins	AliExpress/Amazon: "0.96 SSD1306 I2C OLED", "1.3 SH1106 OLED"
All-in-one pocket	M5Cardputer	esp32s3 (8MB, keyboard)	Built-in screen + keyboard; profile board entry	M5Stack store / Amazon: "M5Cardputer"
All-in-one stick	M5StickC Plus / Plus2	esp32 / esp32s3	Built-in screen + battery	M5Stack / Amazon: "M5StickC Plus2"
Cheap + screen	CYD "Cheap Yellow Display" ESP32-2432S028R	esp32	~US\$10–15 starter with TFT	AliExpress/Amazon: "ESP32-2432S028R CYD 2.8"
High-res screen	LilyGO T-Display-S3	esp32s3	Bright 1.9" ST7789	LilyGO store / Amazon: "LilyGO T-Display-S3"
Flipper add-on	Flipper Zero WiFi Dev Board (ESP32-S2/S3)	esp32s3	Use the f _z build flag (no BT)	Flipper official store; "Flipper WiFi dev board"
Pebble / tiny	M5Atom S3 / S3R	esp32s3	Smallest screened option	M5Stack / Amazon: "M5Atom S3"

Upstream display `#define` options map to these boards: SSD1306, SH1106, SSD1305, WEMOS_OLED_SHIELD, IDEASPARK_SSD1306, CYD, T_DISPLAY_S3, M5STICKCP, M5STICKCP2, M5CARDPUTER, M5ATOMS3, M5ATOMSR3.

Accessories: a quality **USB-C/micro-USB data cable** (not charge-only), a stable **power source** (the upstream prereq calls out "a reliable and appropriate power source"); for an OLED build, **4 jumper wires**

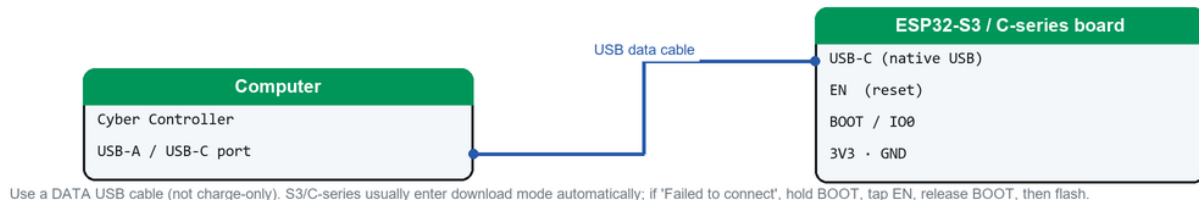
(VCC/ GND/SDA/SCL). **Avoid** ESP8266 and single-core ESP32 modules — they will not run this firmware. Verify the board is genuinely **4MB+** flash (S3 builds expect ~8MB).

4. Building / Assembly

- **Pre-built screened boards (M5Cardputer/StickC, CYD, T-Display-S3):** no assembly — just a data cable. Pick the matching display in the firmware/Cyber Controller or the screen stays blank.
- **DevKit + I²C OLED DIY:** wire OLED **VCC→3V3**, **GND→GND**, **SDA→GPIO21**, **SCL→GPIO22** (typical ESP32 I²C pins — *verify your board's silkscreen*), then enable the screen in config (below).
- **Drivers:** install the USB-serial driver for your board's chip — **CP210x** (most WROOM DevKits), **CH340** (cheap clones), or native USB (S3). Without it, no COM port appears.
- **Firmware "config" matters more than wiring.** Minigotchi behavior is set at compile time in two files (defaults below; the official prebuilt .bin you flash from Cyber Controller bakes in sane defaults — only rebuild from source if you need to change these):
- **config.h** — `#define disp 1` if using any display (0 if headless); `#define fz 1` for Flipper-Zero/no-Bluetooth boards (0 otherwise); set exactly **one** display macro (e.g. `#define SSD1306 1`) to 1 and the rest to 0.
- **config.cpp** — `bool Config::deauth` (default true), `bool Config::advertise` (true), `bool Config::scan` (true, Pwnagotchi scanning), `bool Config::parasite` (false; serial talk to a Pwnagotchi), `bool Config::display + std::string Config::screen`, `int Config::channel` (start channel, 1), `int Config::channels[13]` (hop list 1–13), `int Config::baud` (115200), `int Config::shortDelay` (500) / `longDelay` (5000), and `std::string Config::whitelist[]` (up to ~10 protected SSIDs).
- **Building from source (only if customizing):** Arduino IDE → add the Espressif board-manager URLs → install **esp32 core 2.0.10** (the install guide pins this version) → install libs **ArduinoJson**, **Adafruit GFX**, **AsyncTCP**, **ESPAsyncWebServer (bmorcelli fork, add as .ZIP)** plus the display lib for your screen (Adafruit SSD1306 / u8g2 / TFT_eSPI / M5Unified). The guide also notes a **linker workaround** (add `-zmuldefs`) for certain core versions. Then **Sketch** → **Export Compiled Binary** to get the .bin. *Most users skip all of this and just flash the prebuilt release via Cyber Controller (\$5).*

5. Flashing & First Run (via Cyber Controller)

Flashing connection — ESP32 (native USB)



How to connect the board to flash it via Cyber Controller (native-USB ESP32).

1. Connect the board by USB; open Cyber Controller → **Flash** tab.
2. **Port:** pick the board's COM/tty (click *Refresh* if missing → driver issue).
3. **Firmware Profile:** MinigotchiV3 (dj1ch) (profile id minigotchi-v3).
4. **Board / variant:** choose your board. The resolver maps the release asset name to a chip — names containing cardputer/m5cardputer/s3 → **esp32s3**; cyd/esp32/wroom → **esp32** (default esp32). For a plain WROOM DevKit pick the **ESP32 Dual-Core Generic** entry.
5. Click **Flash**. Because the image model is a **merged single-.bin**, Cyber Controller writes the one file at offset **0x0** (no separate bootloader/partition files) at **115200** baud. First boot starts the minigotchi loop: on a screened board you get the pet face; headless boards print a banner/serial log.
6. **First-run WebUI (whitelist):** on first boot the device raises an AP named **minigotchi** (or minigotchi 2), password **dj1ch-minigotchi**. Join it, browse to **http://192.168.4.1**, enter your protected networks as **SSID, SSID, SSID** in the first box, then **disable the WebUI** by typing **true** in the last box. (verify: WebUI availability depends on the build/release.)

6. Integrate into Cyber Controller

- **Profile:** `minigotchi-v3` (esptool backend; `image_model: merged-single-bin`; `app_offset: 0x0`; `resolver: github_release`; `supports_suicide: false`; `default_baud: 115200`).
- **Release resolution:** the profile resolves the latest GitHub release `.bin` (`asset_match = .bin`). The configured `api_url` is `dj1ch/minigotchi-V3` with `alt_esp32 = dj1ch/minigotchi-ESP32/releases/latest`. Since the **-v3 repo currently 404s**, expect Cyber Controller to fall back to / use the **minigotchi-ESP32** release line — *verify the live repo before relying on auto-fetch (§9)*.
- **Control (serial):** open the **Devices** tab → select the port → **Connect** at **115200** baud. The profile has `protocol: null`, so Cyber Controller acts as a plain serial terminal/monitor for minigotchi — it shows the device's log (channel hops, deauth, advertisement, Pwnagotchi detections) but does **not** parse structured targets the way the Marauder protocol does. Spacehuhn's web Serial Terminal works equally well at 115200.
- **No suicide / no command palette:** `supports_suicide: false` and there is no protocol command set — control is by what the firmware was configured to do at build/first-run (whitelist, parasite, deauth on/off), not by live commands.
- **Backup first:** use **Backup** in the Flash tab to dump current flash before overwriting.

7. Usage (end-to-end)

1. **Power on (authorized lab only).** The minigotchi automatically loops: **channel-hop** → **deauth** → **advertise** → **scan for Pwnagotchi**. On a screened board the face reflects mood/activity.
2. **Whitelist your safe networks** (WebUI at first run, §5, or `Config::whitelist[]` at build) so it never deauths gear you must not touch.
3. **Collect handshakes/PMKID:** the deauth nudges clients to reconnect, producing 4-way handshakes; the material is intended to be cracked by a paired **Pwnagotchi**, not on-device. (*verify exact on-device capture/storage behavior for your release.*)
4. **Talk to friends:** with `advertise = true` it broadcasts its beacon so other minigotchi see it; with `scan = true` it detects nearby Pwnagotchi and shows their stats.
5. **Parasite mode (optional):** set `Config::parasite = true` and wire/serial-connect to a Pwnagotchi (requires the minigotchi Pwnagotchi **plugin**) so the minigotchi syncs over serial as a parasite/companion.
6. **Monitor:** Cyber Controller **Devices** tab (or Serial Monitor) at **115200** to watch the log.
7. **Stop:** Disconnect to free the port; power off the board. To change behavior, edit config and re-flash.

8. Troubleshooting

- **No COM port:** install CP210x/CH340 driver; try another (data) cable; on Linux add your user to `dialout` and replug.
- **"Won't run" / crashes immediately:** confirm a **dual-core ESP32** — single-core ESP32 and **ESP8266** are unsupported by design.
- **Blank/garbled screen:** wrong display selected — the build must have the matching `#define` (e.g. `SSD1306` vs `SH1106` vs `CYD` vs `T_DISPLAY_S3`) set to `1` and `disp 1`; re-flash a build for your exact screen, and check OLED I²C wiring/address.
- **No Bluetooth board (Flipper dev board):** build with `#define fz 1` or BT calls will fault.
- **Flash fails / "Failed to connect":** hold **BOOT** while plugging in (or during connect), lower the flash baud in Settings, and close any serial monitor holding the port.
- **Boot-loops / brick:** use **Erase Flash** then re-flash; the upstream install guide also enables *Erase All Flash Before Sketch Upload*. Verify the board is genuinely 4MB+ (8MB for S3).
- **Can't reach the WebUI:** join AP `minigotchi/minigotchi 2` (pwd `dj1ch-minigotchi`) and browse `http://192.168.4.1`; remember the WebUI is meant to be disabled (`true`) after setup. (*verify per release.*)
- **Auto-fetch fails in Cyber Controller:** the profile's `dj1ch/minigotchi-v3` URL 404s — confirm/repoint to `dj1ch/minigotchi-ESP32` releases (§9) and re-flash from the resolved `.bin`.

9. Sources

- Upstream (live, maintained): <https://github.com/dj1ch/minigotchi-ESP32> — README, `INSTALL.md`, releases (latest `v3.6.4-beta`, 2025-12-23), GPL-3.0. `dj1ch/minigotchi-v3` returns **HTTP 404** at time of writing — the "V3" name tracks the **v3.x release line** of minigotchi-ESP32; **verify** the canonical repo URL.
- Concept/how-it-works wiki (parent project): <https://github.com/dj1ch/minigotchi/wiki/How-the-Minigotchi-works> (channel hop, deauth, `advertise/Frame::advertise()`, `Pwnagotchi::detect()`, `Parasite::readData()`).
- Install/config details: `INSTALL.md` (esp32 core 2.0.10, `display #defines`, `config.cpp/config.h` settings, library list, WebUI AP `minigotchi/dj1ch-minigotchi @ 192.168.4.1`).
- Cyber Controller profile: `src/config/profiles/minigotchi.json` (id `minigotchi`, `core_id/slug minigotchi-v3`, `esptool`, `merged-single-bin @ 0x0`, boards `esp32/esp32s3` incl. `M5Cardputer`, `resolver chip-map`, no suicide).
- Purchasing: M5Stack store, LilyGO store, Flipper store; generic boards on AliExpress/Amazon (ESP32 DevKit V1, ESP32-2432S028R CYD, SSD1306/SH1106 OLED). Verify current board availability + prices at purchase time; vendor links change.